

Contents

ABS	6
ACOS.....	6
ASIN.....	7
ATAN	7
ADDFILE.....	7
ANALOGIN	7
ANALOGOUT	7
BARCODEHEIGHT	8
BARCODETEXT.....	8
BITIN	8
BITOUT	9
BREAKFOR	9
BREAKWHILE	9
CASE	10
CHR.....	10
CLASS.....	11
COMCLEAR	11
COMOPEN	11
COMREAD	11
COMWRITE.....	12
CEIL.....	12
COS.....	12
DATETIME.NOW.....	13
DEFBARCODE	13
DEFFILLPARAM.....	14
DEFFONT	15
DEFIMAGE	15
DEFPARAM	16
DEFPROG.....	16
DEFTTF	17
DRAWARC	17

DRAWCIRCLE	17
DRAWIMAGE	18
DRAWLINE	18
DRAWPOINT	18
DRAWRECT	18
DRAWSPEED	19
DRAWTEXT	19
DRAWWIDTH	19
ELSE	20
ELSEIF	20
ENDCLASS	20
ENDFUNCTION	21
ENDIF	21
ENDSELECT	21
ENDWHILE	22
EXIT	22
FILLSTYLE	22
FILLANGLE	23
FILLGAP	23
FABS	23
FIELDOFFSET	24
FIELDROTATION	24
FIELDSCALE	24
FILECLOSE	25
FILEOPEN	25
FILEREADLN	26
FILESEEK	26
FILESEEKLINE	26
FILEWRITE	26
FILEWRITELN	27
FILLTYPE	27
FOR	28
FUNCTION	28

FLOOR	28
HMICLEAR	29
HMIINPUT	29
HMIMESSAGE.....	29
HMIPRINT.....	30
IF.....	30
IMAGEHEIGHT	30
IMAGEJUSTIFY	30
IMAGESCALEX	31
IMAGESCALEY	31
LASERBEAMON	31
LASERBEAMOFF	31
LASERDUTY.....	32
LASERFREQ.....	32
LASERPOWER	32
LOADPROGFILE.....	32
MIRRORHORIZ.....	32
MIRRORON.....	33
MIRROROFF.....	33
MIRRORVERT.....	33
MOVEABS.....	33
MOVEREL	33
MOVEDIST	33
MOVESPEED	34
MARKFILE	34
NEXT	34
NOT	35
PRIVATE.....	35
PUBLIC.....	35
PI	35
ROUND	36
REMOTEWAIT.....	36
REMOTEREAD.....	36

REMOTEWRITE.....	37
RETURN	37
ROTATEABS	37
ROTATEREL.....	37
RUNPROG.....	38
RUNPROGFILE	38
SELECT	38
SLEEP	39
SIN	39
SQRT.....	39
SPECIALFUNCTION	40
TAN.....	40
TEGTCIRCLE	40
TEXTLINE	41
TEXTJUSTIFYL	41
TEXTJUSTIFYC.....	41
TEXTJUSTIFYR.....	42
TEXTLEFT	42
TEGTCENTER.....	43
TEXTRIGHT	44
TEXTCW	44
TEXTCCW.....	45
TEXTRADIUS	46
TEXTHEIGHT	46
TEXTSCALEX.....	46
TEXTSCALEY.....	47
TEXTHORIZONTAL	47
TEXTVERTICAL	47
TIMECONST	48
TRACECONTOUR	48
TRACEOUTLINE.....	48
UNITSINCH	49
UNITSMM.....	49

USEFILL 49

USEFILLPARAM..... 50

USEPARAM 51

USETTF 51

USEFONT 51

WHILE 52

EXAMPLES 53

Command Reference

ABS

Gets the absolute value

Syntax: ABS arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE var AS DOUBLE
var = 3.14
var = ABS(var)
HMIMESSAGE var.TOSTRING
```

ACOS

Returns arccos

Syntax: ACOS arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE n, d AS DOUBLE
n= 1
d = ACOS (d) * 180 / PI;
HMIMESSAGE ("Acos of " + n.TOSTRING + " is " + d.TOSTRING + " degrees")
```

ASIN

Returns arcSin

Syntax: ASIN arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE n, d AS DOUBLE
n= 1
d = ASIN (d) * 180 / PI;
HMIMESSAGE ("Asin of " + n.TOSTRING + " is " + d.TOSTRING + " degrees")
```

ATAN

Returns arcTan

Syntax: ATAN arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE n, d AS DOUBLE
n= 1
d = ATAN (d) * 180 / PI;
HMIMESSAGE ("Atan of " + n.TOSTRING + " is " + d.TOSTRING + " degrees")
```

ADDFILE

References another file

Syntax: ADDFILE "filename"

filename is character string

```
PRIVATE var AS STRING
var = "Hello World"

ADDFILE "File.Fmp"

;'File.Fmp'

HMIMESSAGE var
```

ANALOGIN

Analog input values

ANALOGOUT

Analog output values

BARCODEHEIGHT

Height for marking barcode

Syntax: BARCODEHEIGHT arg1,arg2

arg1 barcode and arg2 are real numbers (datatypes : INTEGER, DOUBLE)

```
UNITSM
DEFBARCODE 1, "2D,10,10,,,1,,,,,1,0.1,50"

MOVEABS 0 0
BARCODETEXT 1 "abc123"

BARCODEHEIGHT 1 25.4
DRAWBARCODE 1
```

BARCODETEXT

Text for marking barcode

Syntax: BARCODETEXT arg1,arg2

arg1 barcode is real number and arg2 is character string (datatypes : INTEGER, STRING)

```
UNITSM
DEFBARCODE 1, "2D,10,10,,,1,,,,,1,0.1,50"

MOVEABS 0 0
BARCODETEXT 1 "abc123"

BARCODEHEIGHT 1 25.4
DRAWBARCODE 1
```

BITIN

Check digital input values

Syntax: BITIN arg1,arg2

arg1 bit number and arg2 value are real numbers (datatypes : INTEGER, INTEGER)

```
IF BITIN(4, 1)
    HMIMESSAGE "Input 4 is On"

ELSE
    HMIMESSAGE "Input 4 is Off"

ENDIF
```


BITOUT

Set or clear digital output values

Syntax: BITOUT arg1, arg2

arg1 bit number and arg2 value are real numbers (datatypes : INTEGER, INTEGER)

```
BITOUT 2, 1
HMIMESSAGE "Output 2 On"

BITOUT 2, 0
HMIMESSAGE "Output 2 Off"
```

BREAKFOR

Used to break out of FOR NEXT command

```
PRIVATE i,j AS INTEGER
FOR i = 1 TO 10
    j = j + 1
    IF i = 3
        BREAKFOR
    ENDIF
NEXT
HMIMESSAGE j.TOSTRING
```

BREAKWHILE

Used to break out of WHILE command

```
PRIVATE i,j AS INTEGER
i = 1
WHILE i <= 10
    j = j + 1
    SELECT j
        CASE 3
            IF j = 3
                BREAKWHILE
            ENDIF
    ENDSELECT
ENDWHILE
HMIMESSAGE j.TOSTRING
```

CASE

Used with SELECT command when the condition matches

Syntax: CASE arg1

arg1 is a real number (datatype : INTEGER)

```
PRIVATE i,j AS INTEGER
FOR i = 1 to 10
  j = j + 1
  SELECT j
    CASE 3
      HMIMESSAGE j.TOSTRING
    ENDSELECT
NEXT
HMIMESSAGE j.TOSTRING
```

CHR

Labels in FVF file for each letter

Syntax: CHR arg1

arg1 is a real number (datatype :INTEGER)

```
DEFFONT 2, "MyFont.Fvf"

USEFONT 2
TEXTHEIGHT .1
DRAWTEXT "A"
```

CLASS

Gives section of program a name

Syntax: CLASS arg1

arg1 is a character string (datatype : STRING)

```
PRIVATE var AS STRING
var = "Hello"

foo.set(var)
HMIMESSAGE(Foo.Get())

CLASS foo
PRIVATE value AS STRING
FUNCTION set(BYVAL newvalue AS STRING)
    value = newvalue
ENDFUNCTION
FUNCTION get() AS STRING
    RETURN value
ENDFUNCTION
ENDCLASS
```

COMCLEAR

Clears the serial port

Syntax: COMCLEAR arg1

arg1 is a real number (datatype : INTEGER)

```
COMCLEAR 1
```

COMOPEN

Opens the serial port

Syntax: COMOPEN arg1

arg1 is a real number (datatype : INTEGER)

```
COMOPEN 1, "9600,N,8,1"
```

COMREAD

Gets value from the serial port

Syntax: COMREAD arg1, arg2

arg1 is a real number and arg2 is a character string (datatypes : INTEGER,STRING)

```
COMREAD 1,var
```

COMWRITE

Puts a value to serial port

Syntax: COMWRITE arg1, arg2

arg1 is a real number and arg2 is a character string (datatypes : INTEGER,STRING)

```
PRIVATE var AS STRING
COMWRITE 1, "Hello\r"

COMREAD 1, var, 4000
HMIMESSAGE var
```

CEIL

Rounds up a value

Syntax: CEIL arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE var AS DOUBLE
var = CEIL(4.567)
HMIMESSAGE var.TOSTRING
```

COS

Gets COS

Syntax: COS arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE var AS DOUBLE
var = PI
var = COS(var)
HMIMESSAGE var.TOSTRING
```

DATETIME.NOW

Gets a computer' date and time

Syntax: DATETIME.NOW

```
DATETIME .NOW
HMIMESSAGE DATETIME .HOURS
HMIMESSAGE DATETIME .MINUTES
HMIMESSAGE DATETIME .SECONDS
HMIMESSAGE DATETIME .HOURS24
HMIMESSAGE DATETIME .DAY
HMIMESSAGE DATETIME .DAY2
HMIMESSAGE DATETIME .DAY3
HMIMESSAGE DATETIME .MONTH
HMIMESSAGE DATETIME .MONTH1
HMIMESSAGE DATETIME .MONTH2
HMIMESSAGE DATETIME .MONTH3
HMIMESSAGE DATETIME .YEAR
HMIMESSAGE DATETIME .YEAR4
```

DEFBARCODE

Barcoding information

Syntax: DEFBARCODE arg1 arg2

arg1 parameter is real number and arg2 is character string (datatypes : INTEGER,STRING)

Type **UPCA UPCE EAN8 EAN13 39 128 2D QR** followed by type parameters

Height,Width,NarrowBar,NarrowSpace,ModuleHeight,Notch,Border,Point,Duration,Polarity,

QuietZone,Fill,BeamSize,Overlap,CheckDig,2DFormat,2DSize,2DRect,Codesize,Format,Ecc,Mask"

UNITSM

MOVEABS 0 0

DEFBARCODE 1, "2D,10,10,,,1,,,,,1,0.1,50"

BARCODETEXT 1 "abc123"

BARCODEHEIGHT 1 25.4

DRAWBARCODE 1

DEFFILLPARAM

File which contains optional lasing parameters when filling

Syntax: DEFFILLPARAM arg1 arg2

arg1 filename is a real number and arg2 is a character string (datatypes : INTEGER,STRING)

```
DEFTTF 1, "arial"

FILLSTYLE 1, 0
FILLANGLE 1, 0
FILLGAP 1, 0.01
FILLOUTERGAP 1, 0
FILLINNERGAP 1, 0
USEFILL 1
USETTF 1

FILLTYPE 2
DEFFILLPARAM 1, "parameters.fmp"

USEFILLPARAM 1
TEXTHEIGHT .1
DRAWTEXT "OXOXOXOX"

;'Parameters.Fmp'

DRAWSPEED 10
```

DEFFONT

File which contains character lasing information when marking text without true types

Syntax: DEFFONT arg1 arg2

arg1 is a real number and arg2 filename is a character string (datatypes : INTEGER,STRING)

```
DEFFONT 2, "MyFont.fvf"
```

```
USEFONT 2  
TEXTHEIGHT .1  
DRAWTEXT "A"
```

DEFIMAGE

File which contains graphic lasing information when marking an image

Syntax: DEFIMAGE arg1 arg2

arg1 filename is a real number and arg2 is a character string (datatypes : INTEGER,STRING)

```
MOVEREL 0, 2  
DEFIMAGE 1, "mylogo.fvi"
```

```
DRAWIMAGE 1  
MOVEREL 0, -4  
IMAGEHEIGHT 2  
DRAWIMAGE 1
```

DEFPARAM

File which contains optional lasing parameters when non filling

Syntax: DEFPARAM arg1 arg2

arg1 filename is a real number and arg2 is a character string (datatypes : INTEGER,STRING)

```
DEFTTF 1, "arial"

FILLSTYLE 1, 0
FILLANGLE 1, 0
FILLGAP 1, 0.01
FILLOUTERGAP 1, 0
FILLINNERGAP 1, 0
USEFILL 1
USETTF 1

FILLTYPE 2
DEFPARAM 2, "parameters.fmp"

USEPARAM 2
TEXTHEIGHT .1
DRAWTEXT "OXOXOXOX"

;'Parameters.Fmp'

DRAWSPEED 10
```

DEFPROG

File which contains commands

Syntax: DEFPROG arg1 arg2

arg1 is a real number and arg2 filename is a character string (datatypes : INTEGER,STRING)

```
DEFPROG 1, "MyFile.Fmp"

RUNPROG 1

;'MyFile.Fmp'

PRIVATE var AS STRING
var = "Hello"

HMIMESSAGE var
```


DEFTTF

File which contains character lasing information when marking text with true type font

Syntax: DEFTTF arg1 arg2

arg1 is a real number and arg2 filename is a character string (datatypes : INTEGER,STRING)

```
DEFTTF 1, "arial"
```

```
USETTF 1
```

```
TEXTHEIGHT .1
```

```
DRAWTEXT "OXOXOXOX"
```

DRAWARC

Lases an arc relative to current marking position

Syntax: DRAWARC x,y,i,j

args are real numbers (datatypes : DOUBLE,DOUBLE,DOUBLE,DOUBLE)

```
DRAWARC 1 1 1 0
```

```
DRAWARC 1 -1 0 -1
```

```
DRAWARC -1 -1 -1 0
```

```
DRAWARC -1 1 0 1
```

DRAWCIRCLE

Lases a circle relative to current marking position defined by its radius

Syntax: DRAWCIRCLE arg1

arg1 radius is a real number (datatype :DOUBLE)

```
DRAWCIRCLE 1
```

DRAWIMAGE

Lases the image defined by its FVI image file

Syntax: DEFIMAGE arg1 arg2

arg1 is a real number and arg2 is a character string (datatypes : INTEGER,STRING)

```
DEFIMAGE 1, "mylogo.fvi"
```

```
MOVEREL 0, 1  
IMAGEHEIGHT 1  
DRAWIMAGE 1
```

```
MOVEREL 0, 2  
IMAGEHEIGHT 2  
DRAWIMAGE 1
```

DRAWLINE

Lases a line relative to current marking position

Syntax: DRAWLINE x,y

args are real numbers (datatypes : DOUBLE,DOUBLE)

```
DRAWLINE 1, 1
```

DRAWPOINT

Lases a fine beam current to marking position

Syntax: DRAWPOINT arg1

arg1 is a real number (datatype : DOUBLE)

```
DRAWPOINT 1
```

DRAWRECT

Lases rectangle defined by its length and width

Syntax: DRAWRECT arg1, arg2

arg1 width and arg2 height are real numbers (datatypes: DOUBLE, DOUBLE)

```
DRAWRECT 1, 1
```

DRAWSPEED

Commands the speed for lasing

Syntax: DRAWSPEED arg1

arg1 speed is a real number (datatype: DOUBLE)

```
UNITSINCH
DRAWSPEED 1
UNITSMM
DRAWSPEED 25.4
DRAWRECT 25.4, 25.4 ;4 Seconds
```

DRAWTEXT

Commands laser to lase text

Syntax: DRAWTEXT arg1

arg1 is a character string (datatype:STRING)

```
DRAWTEXT "Hello"
```

DRAWWIDTH

Commands circles to be lased and distance between circles

Syntax: DRAWWIDTH arg1,arg2

arg1 and arg2 are real numbers (datatypes:DOUBLE,DOUBLE)

```
DRAWWIDTH 0.1, 0.1
DRAWRECT 1,1 ;Thick Rectangle
```

ELSE

Used with IF command when all other conditions dont match

Syntax: ELSE

```
IF 1 = 2
    HMIMESSAGE "No"

ELSE
    HMIMESSAGE "Yes"

ENDIF
```

ELSEIF

Used with IF command when a condition matches

Syntax: ELSEIF value

value is a real number or character string (datatype:INTEGER,DOUBLE,STRING)

```
IF 1 = 2
    HMIMESSAGE "No"

ELSEIF 1 = 1
    HMIMESSAGE "Yes"

ENDIF
```

ENDCLASS

Used to mark end of CLASS

Syntax: ENDCLASS

```
CLASS foo
PRIVATE value AS STRING
FUNCTION set(BYVAL newvalue AS STRING)
    value = newvalue
ENDFUNCTION
FUNCTION get() AS STRING
    RETURN value
ENDFUNCTION
ENDCLASS
```

ENDFUNCTION

Used to mark end of FUNCTION

Syntax: ENDFUNCTION

```
PRIVATE x,y AS DOUBLE
x = 9
y = f(x)
HMIMESSAGE y.TOSTRING
FUNCTION f(BYVAL x AS DOUBLE)
RETURN SQR(x)
ENDFUNCTION
```

ENDIF

Used to mark end of IF command

Syntax: ENDIF

```
IF 1 = 1
    HMIMESSAGE "Yes"
ENDIF
```

ENDSELECT

Used to mark end of SELECT command

Syntax: ENDSELECT

```
PRIVATE i,j AS INTEGER
FOR i = 1 to 10
    j = j + 1
    SELECT j
    CASE 3
        HMIMESSAGE j.TOSTRING
    ENDSELECT
NEXT
```

ENDWHILE

Used to mark end of WHILE command

Syntax: ENDWHILE

```
PRIVATE i,j AS INTEGER
i = 1
WHILE i <= 10
    j = j + 1
    SELECT j
        CASE 3
            IF j = 3
                BREAKWHILE
            ENDIF
        ENDSELECT
    ENDWHILE
    HMIMESSAGE j.TOSTRING
```

EXIT

Aborts program

Syntax: EXIT

```
HMIMESSAGE "Yes"

EXIT
HMIMESSAGE "No"
```

FILLSTYLE

Style of hatching pattern

Syntax: FILLSTYLE arg1 arg2

arg1 pattern number and arg2 are real numbers (datatypes : INTEGER,DOUBLE)

```
DEFTTF 1, "arial"

USETTF 1
FILLSTYLE 1, 0
USEFILL 1
FILLTYPE 2
DRAWTEXT "OXOXOXOX"
```

FILLANGLE

Angle of hatching pattern

Syntax: FILLANGLE arg1 arg2

arg1 pattern number and arg2 are real numbers (datatypes : INTEGER,DOUBLE)

```
DEFTTF 1, "arial"  
  
USETTF 1  
FILLANGLE 1, 45  
USEFILL 1  
FILLTYPE 2  
DRAWTEXT "OXOXOXOX"
```

FILLGAP

Offsets of hatching pattern

Syntax: FILLGAP arg1 arg2

arg1 pattern number and arg2 are real numbers (datatypes : INTEGER,DOUBLE)

```
DEFTTF 1, "arial"  
  
USETTF 1  
FILLGAP 1, 0.5  
USEFILL 1  
FILLTYPE 2  
DRAWTEXT "OXOXOXOX"
```

FABS

Gets the absolute value

Syntax: FABS arg1

arg1 is a real number (datatype:DOUBLE)

```
PRIVATE var AS DOUBLE  
var = -1.234  
var = FABS(var)  
HMIMESSAGE var.TOSTRING
```

FIELDOFFSET

Used to offset absolute positioning when marking

Syntax: FIELDOFFSET arg1,arg2

arg1 x and arg2 y are real numbers (datatypes:DOUBLE,DOUBLE)

```
FIELDOFFSET 1 1
MOVEABS 1 1
MOVEREL 1 1
MOVEDIST 1 1
HMIMESSAGE "Position=4,4"
```

FIELDROTATION

Used to offset rotation of absolute marking

Syntax: FIELDROTATION arg1

arg1 degrees is a real number (datatype:DOUBLE)

```
FIELDROTATION 90
MOVEABS .1 .1
MOVEREL .1 .1
MOVEDIST .1 .1
HMIMESSAGE "Position=.3,-.3"
```

FIELDSCALE

Used to reduce or enlarge all marking by a percentage

Syntax: FIELDSCALE arg1

arg1 percentage is a real number (datatype:DOUBLE)

```
FIELDSCALE 200
MOVEABS .1 .1
MOVEREL .1 .1
MOVEDIST .1 .1
HMIMESSAGE "Position=.6,.6"
```


FILECLOSE

Closes association to a text file during program run

Syntax: FILECLOSE arg1

arg1 is a real number (datatype:INTEGER)

```
PRIVATE f AS INTEGER
```

```
PRIVATE s AS STRING
```

```
FILEOPEN(f, "myfile.txt", "w")
```

```
FILEWRITELN("LaserStar", f)
```

```
FILEWRITELN(" Technologies", f)
```

```
FILEWRITELN("Corporation", f)
```

```
FILECLOSE(f)
```

FILEOPEN

Opens association to a text file for reading and writing during program run

Syntax: FILEOPEN arg1,arg2,arg3

arg1 is a real number and arg2 and 3 are character strings (datatypes (INTEGER,STRING,STRING))

```
PRIVATE f AS INTEGER
```

```
PRIVATE s AS STRING
```

```
FILEOPEN(f, "myfile.txt", "w")
```

```
FILEWRITELN("LaserStar", f)
```

```
FILEWRITELN("Technologies", f)
```

```
FILEWRITELN("Corporation", f)
```

```
FILECLOSE(f)
```

FILEREADLN

Reads file when the program is run

Syntax: FILEREADLN arg1,arg2

arg1 is a character string and arg2 is a real number (datatypes:STRING,INTEGER)

```
PRIVATE f AS INTEGER
PRIVATE s AS STRING
FILEOPEN(f, "myfile.txt", "r")

FILEREADLN(s,f)
FILEREADLN(s,f)
FILEREADLN(s,f)
FILECLOSE(f)
```

FILESEEK

Positions to where file will be read next during program

Syntax: FILESEEK arg1,arg2

arg1 is a real number and arg2 is a character string (datatypes : (INTEGER,STRING)

```
FILESEEK(2,f)
```

FILESEEKLINE

Positions to a start of line number in file for reading during program

Syntax: FILESEEKLINE arg1,arg2

arg1 is a real number and arg2 is a character string (datatypes : INTEGER,STRING)

```
FILESEEKLINE(2,f)
```

FILEWRITE

Writes to file when the program is run

Syntax: FILEWRITE arg1,arg2

arg1 is a character string and arg2 is a real number (datatypes = STRING,INTEGER)

```
PRIVATE f AS INTEGER
FILEOPEN(f,"myfile.txt","w")

FILEWRITE("LaserStar",f)
FILEWRITE(" Technologies",f)
FILECLOSE(f)
```

FILEWRITELN

Writes to file followed by end of line character when program is run

Syntax: FILEWRITELN arg1,arg2

arg1 is a character string and arg2 is a real number (datatypes = STRING,INTEGER)

```
PRIVATE f AS INTEGER
FILEOPEN(f,"myfile.txt", "w")

FILEWRITELN("LaserStar",f)

FILEWRITELN("Technologies",f)

FILEWRITELN("Corporation",f)

FILECLOSE(f)
```

FILLTYPE

Used when hatching to choose lasing outline before fill

Syntax: FILLTYPE arg1

arg1 is a real number (datatype : INTEGER)

```
UNITSINCH
DEFTTF 3, "arial"

USETTF 3
TEXTHEIGHT .1
FILLANGLE 2, 45
USEFILL 2
FILLTYPE 2
DRAWTEXT "OXOXOXOX"
```

FOR

Command loop count around commands

Syntax: FOR value

value is real number to real number (datatype : INTEGER to INTEGER)

```
UNITSINCH
PRIVATE x, y AS DOUBLE
PRIVATE i, j AS INTEGER
x = 0
y = 2
FOR i = 1 TO 4
    x = 0
    FOR j = 1 TO 3
        MOVEABS x y
        x = x + .5
    NEXT
    y = y - 1.5
NEXT
```

FUNCTION

Used within a program to create reference to a routine

Syntax: FUNCTION values

values are real numbers or character strings (datatypes = INTEGER DOUBLE or STRING)

```
PRIVATE x,y AS DOUBLE
x = 9
y = f(x)
FUNCTION f(BYVAL x AS DOUBLE)
    RETURN 1
ENDFUNCTION
```

FLOOR

Rounds down the value

Syntax: FLOOR arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE var AS DOUBLE
var = FLOOR(4.567)
HMIMESSAGE var.TOSTRING
```

HMICLEAR

During program run clears the operator screen

Syntax: HMICLEAR

```
HMIPRINT 1, 1, "Hello World"
```

```
HMICLEAR
```

HMIINPUT

During program run waits for operator to reply

Syntax: HMIINPUT arg1

arg1 is a character string (datatype : STRING)

```
PRIVATE var AS STRING
```

```
HMIPRINT 1, 1, "Hit any key to continue"
```

```
HMIINPUT var
```

```
HMIPRINT 2, 1, var
```

HMIMESSAGE

During program a popup message displays and waits for operator to reply

Syntax: HMIMESAGE arg1

arg1 is a character string (datatype : STRING)

```
HMIMESSAGE "Hit any key to continue"
```

HMIPRINT

During program run message displays to operator screen without reply

Syntax: HMIPRINT arg1, arg2, arg3

arg1 and arg2 are row and column are real numbers and arg3 is a character string

(datatypes : INTEGER,INTEGER,STRING)

```
HMIPRINT 1, 1, "Hello World"
```

IF

Used in making decisions logical to your program

Syntax: IF value

value is a real number or character string (datatype : INTEGER or STRING)

```
IF 1 = 1
    HMIMESSAGE "Ok"
ENDIF
```

IMAGEHEIGHT

Marking size for a file which contains graphic lasing information

Syntax: IMAGEHEIGHT arg1

arg1 is a real number (datatype : DOUBLE)

```
DEFIMAGE 42, "mylogo.fvi"
IMAGEHEIGHT 2
DRAWIMAGE 42
```

IMAGEJUSTIFY

FVI file marking justification Top 1,2,3 Middle 4,5,6 or Bottom 7,8,or 9

Syntax: IMAGEJUSTIFY arg1

arg1 is a real number (datatype : INTEGER)

```
DEFIMAGE 2, "mylogo.fvi"
IMAGEHEIGHT 2
IMAGEJUSTIFY 0 ;non justification
DRAWIMAGE 2
```

IMAGESCALEX

FVI file marking width adjustment here

Syntax: IMAGESCALEX arg1

arg1 is a real number (datatype : DOUBLE)

```
DEFIMAGE 2, "mylogo.fvi"
```

```
IMAGEHEIGHT 2
```

```
IMAGESCALEX 50
```

```
DRAWIMAGE 2
```

IMAGESCALEY

FVI file marking height adjustment here

Syntax: IMAGESCALEY arg1

arg1 is a real number (datatype : DOUBLE)

```
DEFIMAGE 2, "mylogo.fvi"
```

```
IMAGEHEIGHT 2
```

```
IMAGESCALEY 50
```

```
DRAWIMAGE 2
```

LASERBEAMON

Enables fine beam current to marking position

Syntax: LASERBEAMON

LASERBEAMOFF

Disables fine beam current to marking position

Syntax: LASERBEAMOFF

LASERDUTY

Percentage of lasing pulse

Syntax: LASERDUTY arg1

arg1 is a real number (datatype : DOUBLE)

```
LASERDUTY 50
```

LASERFREQ

Lasing pulse

Syntax: LASERFREQ arg1

arg1 is a real number (datatype : DOUBLE)

```
LASERFREQ 50
```

LASERPOWER

Lasing power

Syntax: LASERPOWER arg1

arg1 is a real number (datatype : DOUBLE)

```
LASERPOWER 50
```

LOADPROGFILE

FMP file loaded during runtime

Syntax: LOADPROGFILE arg1, arg2

arg1 is a real number and arg2 filename is a character string

(datatypes : INTEGER,STRING)

```
LOADPROGFILE 1, "Test.fmp"
```

```
RUNPROGFILE 1  
;'Test.Fmp'
```

```
HMIMESSAGE "Test"
```

MIRRORHORIZ

Used to lase the reverse effect in the X Axis

Syntax: MIRRORHORIZ

```
MIRRORHORIZ  
DRAWTEXT "Left"
```


MIRRORON

Used to lase the reverse effect in the Y Axis

Syntax: MIRRORON

MIRROROFF

Used to lase the normal effect

Syntax: MIRROROFF

MIRRORVERT

Used to lase the vertical effect

Syntax: MIRRORVERT

```
MIRRORVERT  
DRAWTEXT "Down"
```

MOVEABS

Used to move position absolutely before marking

Syntax: MOVEABS arg1, arg2

arg1 x and arg2 y are real numbers (datatypes : DOUBLE,DOUBLE)

```
MOVEABS 1 1
```

MOVEREL

Used to move position relatively before marking

Syntax: MOVEREL arg1, arg2

arg1 x and arg2 y are real numbers (datatypes = DOUBLE,DOUBLE)

```
MOVEABS 1 1  
MOVEREL 1 1 ;;;;;;;;;;;;;;Position=2,2
```

MOVEDIST

Used to move position relatively before marking

Syntax: MOVEDIST arg1, arg2

arg1 x and arg2 y are real numbers (datatypes : DOUBLE,DOUBLE)

```
MOVEABS 1 1  
MOVEREL 1 1  
MOVEDIST 1 1 ;;;;;;;;;;;;;;Position=3,3
```

MOVESPEED

Commands the speed for moving

Syntax: MOVESPEED arg1

arg1 is a real number (datatype : DOUBLE)

```
UNITSINCH  
MOVESPEED 1
```

```
UNITSM  
MOVESPEED 25.4
```

MARKFILE

Marks an EZD file

```
MARKFILE "-e1 C:\\LaserStar\\FiberStar\\Sample.Ezd"
```

NEXT

Marks end of repetition in FOR loop

```
UNITSINCH  
PRIVATE x, y AS DOUBLE  
PRIVATE i, j AS INTEGER  
  
x = 0  
y = 2
```

```
FOR i = 1 TO 4  
  x = 0  
  FOR j = 1 TO 3  
    MOVEABS x y  
    DRAWTEXT "+"  
  
    x = x + .5  
  NEXT  
  y = y - 1.5  
NEXT
```

NOT

RETURN 0 if NOT 0 otherwise 1

Syntax: NOT arg1

arg1 is a real number (datatype : INTEGER)

```
PRIVATE Count, Done AS INTEGER
Done = 0
Count = 0
WHILE NOT Done
    Count = Count + 1
    IF (Count = 3)
        Done = 1
    ENDIF
ENDWHILE
```

PRIVATE

Variable has local scope applicable to variable types

```
PRIVATE var AS STRING
var = "Hello"
```

PUBLIC

Variable has global scope applicable to variable types

```
PUBLIC var AS STRING
var = "Hello"

DEFPROG 1, "MyFile.Fmp"

RUNPROG 1

;'MyFile.Fmp'

HMIMESSAGE var
```

PI

Programs the value of 3.141593

Syntax: PI

ROUND

Rounds real value up or down

Syntax: ROUND arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE var AS DOUBLE
var =2.5
var = ROUND(var)
HMIMESSAGE var.TOSTRING
```

REMOTEWAIT

Gets value from the TCP Port with suspending program

Syntax: REMOTEWAIT arg1

arg1 is a character string (datatype : STRING)

```
PRIVATE var AS INTEGER
PRIVATE msg AS STRING
IF (var == 1)
    msg = "Engraver"

    REMOTEWAIT msg
    var = 0
ENDIF
```

REMOTEREAD

Gets value from the TCP Port without suspending program

Syntax: REMOTEREAD arg1,arg2

arg1 is a real number and arg2 is a character string (datatypes : INTEGER,STRING)

```
PRIVATE var AS STRING
WHILE 1
    var = ""

    REMOTEREAD 1, var, 1000
    IF var <> ""

        BREAKWHILE
    ENDIF
ENDWHILE
```

REMOTEWRITE

Puts value to the TCP Port

Syntax: REMOTEWRITE arg1,arg2

arg1 is a real number and arg2 is a character string (datatypes : INTEGER,STRING)

```
PRIVATE var AS INTEGER
PRIVATE msg AS STRING
var = 1
msg = "Ready " + var.TOSTRING

REMOTEWRITE 1, msg
```

RETURN

RETURN from FUNCTION with return value

Syntax: RETURN value

value is a real number or character string (datatypes :INTEGER,DOUBLE,STRING)

```
PRIVATE x,y AS DOUBLE
x = 9
x = f(x) ;;;;;;;;;;;;;;0
HMIMESSAGE y.TOSTRING
FUNCTION f(BYVAL x AS DOUBLE)
RETURN SQR(x)
ENDFUNCTION
```

ROTATEABS

Used to rotate absolute marking

Syntax: ROTATEABS arg1

arg1 degrees is a real number (datatype : DOUBLE)

```
ROTATEABS 45
"xxxxxxxxxxxxx" ;Lases at 45 degree angle
```

ROTATEREL

Used to rotate relative marking

Syntax: ROTATEREL arg1

arg1 degrees is a real number (datatype : DOUBLE)

```
ROTATEABS 45
ROTATEREL 45
"xxxxxxxxxxxxx" ;;;;;;;;;;;;;;Lases at 90 degree angle
```

RUNPROG

Used to execute another program referenced by DEFPROG

Syntax: RUNPROG arg1

arg1 is a real number (datatype : INTEGER)

```
DEFPROG 1, "MyFile.Fmp"
```

```
RUNPROG 1
```

```
; 'MyFile.Fmp'
```

```
PRIVATE var AS STRING  
var = "Hello"
```

RUNPROGFILE

Used to execute another program referenced by LOADPROGFILE

Syntax: RUNPROGFILE arg1

arg1 is a real number (datatype : INTEGER)

```
LOADPROGFILE 1, "Test.fmp"
```

```
RUNPROGFILE 1
```

```
; 'Test.Fmp'
```

```
HMIMESSAGE "Test"
```

SELECT

Used in making decisions logical to your program

Syntax: SELECT arg1

arg1 is a real number (datatype : INTEGER)

```
PRIVATE i,j AS INTEGER  
FOR i = 1 to 10  
    j = j + 1  
    SELECT j  
        CASE 3  
            HMIMESSAGE j.TOSTRING  
        ENDSELECT  
NEXT  
HMIMESSAGE j.TOSTRING
```

SLEEP

Suspend running program by number of milliseconds

Syntax: SLEEP arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE i AS INTEGER
FOR i = 1 to 6
  SLEEP 10
NEXT
```

SIN

Gets sin

Syntax: SIN arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE var AS DOUBLE
var = SIN(PI)
HMIMESSAGE var.TOSTRING
```

SQRT

Gets square root of a number

Syntax: SQRT arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE x,y AS DOUBLE
x = 9
y = f(x)
HMIMESSAGE y.TOSTRING
FUNCTION f(BYVAL x AS DOUBLE)
RETURN SQRT(x)
ENDFUNCTION
```

SPECIALFUNCTION

Special FUNCTION without return to automatically get called at beginning and end of program

```
SPECIALFUNCTION 1, 0, Begin
SPECIALFUNCTION 1, 1, End

"This is Program"

FUNCTION Begin()
    HMIMESSAGE "Before Program"

ENDFUNCTION

FUNCTION End()
    HMIMESSAGE "After Program"

ENDFUNCTION
```

TAN

Gets tan

Syntax: TAN arg1

arg1 is a real number (datatype : DOUBLE)

```
PRIVATE var AS DOUBLE
var = TAN(0.7071)
HMIMESSAGE var.TOSTRING
```

TEXCIRCLE

Used to mark polar coordinate system

Syntax: TEXTCIRCLE

```
TEXCIRCLE
TEXTHEIGHT 0.1
TEXTRADIUS 1
ROTATEABS 0
DRAWTEXT "12"

ROTATEABS 90
DRAWTEXT "3"

ROTATEABS 180
DRAWTEXT "6"

ROTATEABS 270
DRAWTEXT "9"
```


TEXTLINE

Used to mark non polar coordinate system same as default

Syntax: TEXTLINE

```
TEXCIRCLE  
TEXTHEIGHT 0.1  
TEXTRADIUS 1  
ROTATEABS 0  
DRAWTEXT "12"
```

```
ROTATEABS 90  
DRAWTEXT "3"
```

```
ROTATEABS 180  
DRAWTEXT "6"
```

```
ROTATEABS 270  
DRAWTEXT "9"
```

```
TEXTLINE  
"Hello World"
```

TEXTJUSTIFYL

Used to justify mark to left when in TEXTLINE

Syntax: TEXTJUSTIFYL arg1

arg1 is a real number (datatype : DOUBLE)

```
TEXTLINE  
TEXTJUSTIFYL 1  
"Hello World"
```

TEXTJUSTIFYC

Used to justify mark to center when in TEXTLINE

Syntax: TEXTJUSTIFYC arg1

arg1 is a real number (datatype : DOUBLE)

```
TEXTLINE  
TEXTJUSTIFYC 1  
"Hello World"
```

TEXTJUSTIFYR

Used to justify mark to right when in TEXTLINE

Syntax: TEXTJUSTIFYR arg1

arg1 is a real number (datatype : DOUBLE)

```
TEXTLINE
TEXTJUSTIFYR 1
"Hello World"
```

TEXTLEFT

Used to justify degrees to left when in TEXTCIRCLE

Syntax: TEXTLEFT arg1

arg1 is a real number (datatype : DOUBLE)

```
TEXTCIRCLE
TEXTCW
TEXTHEIGHT 0.1
EXTRADADIUS 1
ROTATEABS 0
TEXTLEFT 45
DRAWTEXT "12"

ROTATEABS 90
TEXTLEFT 45
DRAWTEXT "3"

ROTATEABS 180
TEXTLEFT 45
DRAWTEXT "6"

ROTATEABS 270
TEXTLEFT 45
DRAWTEXT "9"
```

TEXTCENTER

Used to justify degrees to center when in TEXTCIRCLE

Syntax: TEXTCENTER arg1

arg1 is a real number (datatype : DOUBLE)

```
TEXTCIRCLE  
TEXTCW  
TEXTHEIGHT 0.1  
TEXTRADIUS 1  
ROTATEABS 0  
TEXTCENTER 45  
DRAWTEXT "12"
```

```
ROTATEABS 90  
TEXTCENTER 45  
DRAWTEXT "3"
```

```
ROTATEABS 180  
TEXTCENTER 45  
DRAWTEXT "6"
```

```
ROTATEABS 270  
TEXTCENTER 45  
DRAWTEXT "9"
```

TEXTRIGHT

Used to justify degrees to right when in TEXTCIRCLE

Syntax: TEXTRIGHT arg1

arg1 is a real number (datatype : DOUBLE)

```
TEXTCIRCLE  
TEXTCW  
TEXTHEIGHT 0.1  
TEXTRADIUS 1  
ROTATEABS 0  
TEXTRIGHT 45  
DRAWTEXT "12"
```

```
ROTATEABS 90  
TEXTRIGHT 45  
DRAWTEXT "3"
```

```
ROTATEABS 180  
TEXTRIGHT 45  
DRAWTEXT "6"
```

```
ROTATEABS 270  
TEXTRIGHT 45  
DRAWTEXT "9"
```

TEXTCW

Used to rotate degrees clockwise when in TEXTCIRCLE

Syntax: TEXTCW

```
TEXTCIRCLE  
TEXTCW  
TEXTHEIGHT 0.1  
TEXTRADIUS 1  
ROTATEABS 0  
DRAWTEXT "12"
```

```
ROTATEABS 90  
DRAWTEXT "3"
```

```
ROTATEABS 180  
DRAWTEXT "6"
```

```
ROTATEABS 270  
DRAWTEXT "9"
```

TEXTCCW

Used to rotate degrees counter clockwise when in TEXTCIRCLE

Syntax: TEXTCCW

```
TEXTCIRCLE  
TEXTCCW  
TEXTHEIGHT 0.1  
TEXTRADIUS 1  
ROTATEABS 0  
DRAWTEXT "12"
```

```
ROTATEABS 90  
DRAWTEXT "3"
```

```
ROTATEABS 180  
DRAWTEXT "6"
```

```
ROTATEABS 270  
DRAWTEXT "9"
```

TEXTRADIUS

Used to position mark when in TEXTCIRCLE

Syntax: TEXTRADIUS arg1

arg1 is a real number (datatype : DOUBLE)

```
TEXTCIRCLE  
TEXTCW  
TEXTHEIGHT 0.1  
TEXTRADIUS 2  
ROTATEABS 0  
DRAWTEXT "12"  
  
ROTATEABS 90  
DRAWTEXT "3"  
  
ROTATEABS 180  
DRAWTEXT "6"  
  
ROTATEABS 270  
DRAWTEXT "9"
```

TEXTHEIGHT

Gives text marking size here

Syntax: TEXTHEIGHT arg1

arg1 is a real number (datatype : DOUBLE)

```
TEXTHEIGHT 1  
DRAWTEXT "Hello World"
```

TEXTSCALEX

Used to reduce or enlarge text mark width by a percentage

Syntax: TEXTSCALEX arg1

arg1 is a real number (datatype : DOUBLE)

```
TEXTLINE  
TEXTHEIGHT 1  
TEXTSCALEX 50  
DRAWTEXT "Hello World"
```

TEXTSCALEY

Used to reduce or enlarge text mark height by a percentage

Syntax: TEXTSCALEY arg1

arg1 is a real number (datatype : DOUBLE)

```
TEXTLINE
TEXTHEIGHT 1
TEXTSCALEY 50
DRAWTEXT "Hello World"
```

TEXTHORIZONTAL

Used to lase text with normal default when in TEXTLINE

Syntax: TEXTHORIZONTAL

```
TEXTLINE
TEXTHEIGHT 1
TEXTHORIZONTAL
DRAWTEXT "Hello World"
```

TEXTVERTICAL

Used to lase text with vertical effect when in TEXTLINE

Syntax: TEXTVERTICAL

```
TEXTLINE

TEXTHEIGHT 1
TEXTVERTICAL
DRAWTEXT "Down"
```

TIMECONST

Laser beam delay in microseconds

Syntax: TIMECONST arg1,arg2

arg1 number arg2 time are real numbers (datatypes : INTEGER,INTEGER)

TC 0, 1000 ;Wait 1000 before the beam is on at a beginning

TC 2, 1000 ; Wait 1000 before the beam is off at an end

TC 3, 1000; Wait 1000 whenever beam position moved

TC 4, 100; Wait 100 when beam is on and position reverses plus or minus

TC 5, 1000; Wait 1000 for each start of a continuous path

TC 6, 1000; Wait 1000 for each end of a continuous path

TRACECONTOUR

Used to turn off lasing and put in red pointer diode

Syntax: TRACEONTOUR arg1,arg2

arg1 is a real number and arg2 is a character string (datatypes :INTEGER,STRING)

TEXTLINE

TEXTHEIGHT 1

TRACECONTOUR 1, "100" ;Here Trace Speed

DRAWTEXT "Text Information"

TRACEOUTLINE

Used to turn off lasing and put in red pointer diode with repeatable red cross hair

Syntax: TRACEOUTLINE arg1,arg2

arg1 is a real number and arg2 is a character string (datatypes : INTEGER,STRING)

TEXTLINE

TEXTHEIGHT 1

TRACEOUTLINE 1, "100" ;Here Trace Speed

DRAWTEXT "Text Information"

TRACEOUTLINE 0, "100" ;Here Trace Speed as Continuous CrossHair

UNITSINCH

English units used for marking and tracing

Syntax: UNITSINCH

`UNITSINCH`

UNITSMM

Metric units used for marking and tracing

Syntax: UNITSMM

`UNITSMM`

USEFILL

Command enables hatching patterns given by DEFFILL

Syntax: USEFILL arg1

arg1 is a real number (datatype : INTEGER)

```
DEFFILLPARAM 1, "parameters.fmp"
```

```
DEFTTF 1, "arial"
```

```
USETTF 1
```

```
TEXTHEIGHT .1
```

```
FILLGAP 1, 0.01
```

```
USEFILL 1
```

```
FILLTYPE 2
```

```
USEFILLPARAM 1
```

```
DRAWTEXT "OXOXOXOX"
```

```
; 'Parameters.Fmp'
```

```
DRAWSPEED 10
```

USEFILLPARAM

Command enables more lasing parameters when hatching

Syntax: USEFILLPARAM arg1

arg1 is a real number (datatype : INTEGER)

```
DEFFILLPARAM 1, "parameters"
```

```
DEFTTF 1, "arial"
```

```
USETTF 1
```

```
TEXTHEIGHT .1
```

```
FILLGAP 1, 0.01
```

```
USEFILL 1
```

```
FILLTYPE 2
```

```
USEFILLPARAM 1
```

```
DRAWTEXT "OXOXOXOX"
```

```
; 'Parameters.Fmp'
```

```
DRAWSPEED 10
```

USEPARAM

Command enables more lasing parameters for outline of hatch

Syntax: USEPARAM arg1

arg1 is a real number (datatype : INTEGER)

```
DEFPARAM 2, "parameters.fmp"
```

```
DEFTTF 1, "arial"
```

```
USETTF 1
```

```
TEXTHEIGHT .1
```

```
FILLGAP 1, 0.01
```

```
USEFILL 1
```

```
FILLTYPE 2
```

```
USEPARAM 2
```

```
DRAWTEXT "OXOXOXOX"
```

```
; 'Parameters.Fmp'
```

```
DRAWSPEED 10
```

USETTF

Command which enables true type lasing when defined by DEFTTF

Syntax: USETTF arg1

arg1 is a real number (datatype : INTEGER)

```
DEFTTF 3, "arial"
```

```
USETTF 3
```

```
TEXTHEIGHT .1
```

```
DRAWTEXT "OXOXOXOX"
```

USEFONT

Command which enables FVF file characters to lase when defined by DEFFONT

Syntax: USEFONT arg1

arg1 is a real number (datatype : INTEGER)

```
DEFFONT 2, "MyFont.fvf"
```

```
USEFONT 2
```

```
TEXTHEIGHT .1
```

```
DRAWTEXT "A"
```

WHILE

Can command conditional loop around commands therein

Syntax: WHILE value

value is a real number or character string (datatype : INTEGER,DOUBLE,STRING)

```
PRIVATE i,j AS INTEGER
i = 1
WHILE i <= 10
    j = j + 1
    SELECT j
    CASE 3
    IF j = 3
        BreakWHILE
    ENDIF
ENDSELECT
ENDWHILE
HMIMESSAGE j.TOSTRING
```

EXAMPLES

```
UNITSINCH
USEFONT 0
TEXTHEIGHT 1
MOVEABS 0 5
PRIVATE var1 AS DOUBLE
var1 = PI
DRAWTEXT var1
MOVEREL 0, -1
PRIVATE var3 AS DOUBLE
PRIVATE var2 AS DOUBLE
var2 = -1
var3 = ABS(var2)
DRAWTEXT var3
MOVEREL 0, -2
var3 = ABS(-3)
DRAWTEXT var3
var1 = -1.234
var2 = FABS(var1)
MOVEREL 0, -3
DRAWTEXT var2
var2 = 0.5
var3 = ACOS(var2)
MOVEREL 0, -4
DRAWTEXT var3
MOVEREL 0, -5
var2 = 1
var3 = ATAN(var2)
DRAWTEXT var3.toString
MOVEREL 0, -6
var2 = 3*PI/2 ;radians
var3 = SIN(var2)
DRAWTEXT var3
var2 = 1.234
var2 = ROUND(var2,2)
MOVEREL 0, -7
DRAWTEXT var2
var2 = 1.234
var1 = ROUND(var2,2)
MOVEREL 0, -7
DRAWTEXT var1
var3 = Sqrt(9)
MOVEREL 0, -8
DRAWTEXT var3
```

;Example 2

```
UNITSINCH
USEFONT 0
TEXTHEIGHT 1
MOVEABS 0 3
MOVEREL 0, -1
PRIVATE var, var1, S1, S2, var22 AS STRING
S1 = "Hello"

S2 = S1.CONCAT("World") ; Hello World in S2

HMIMESSAGE S2 ;Displays Hello World
S1 = "Hello"

S2 = " World! "

S2 = S1.CONCAT(S2)
HMIMESSAGE S2;Displays Hello World!
S1 = S1 + "World" ;Also can append as follows

HMIMESSAGE S1 ;Displays Hello World
var = "001"

var = var.INCREMENT(1)
HMIMESSAGE var ;Displays 002
PRIVATE var44 AS INTEGER
var22 = "ABCDEF"

var44 = var22.INDEXOF("C")

HMIMESSAGE var44.toString ;Lases 2
var = "Hello World"

PRIVATE var2 AS STRING
var2 = var.LEFT(5)
DRAWTEXT var2 ;Lases Hello
var1="ABC001"

var2=var1.MASKINC("iiiiii",1) ;Six i's for incrementing entire ABC001

HMIMESSAGE var2 ;Display ABC002
var2=var1.MASKINC("iiiccc",1) ;Three 'i's for incrementing only ABC

HMIMESSAGE var2;Display ABD001
var2=var2.MASKINC("iiiccc",1)

MOVEREL 0, -2
DRAWTEXT var2
var2=var2.MASKINC("iiiccc",1)

MOVEREL 0, -3
DRAWTEXT var2
var1 = "HelloWorld"
```

```
var2 = var1.MID(2,2)
MOVEREL 0, -4
DRAWTEXT var2
var=" Hello World Goodbye World"

var2=var.RIGHT(5) ;Retrieve last 5 characters
MOVEREL 0, -5
DRAWTEXT var2
var = "      Hello World      "

var = var.TRIM
DRAWTEXT var ;Lases Hello World
```

;Example 3

UNITSINCH

PRIVATE x, y AS DOUBLE

PRIVATE i, j AS INTEGER

x = 0

y = 2

FOR i = 1 TO 4

 x = 0

 FOR j = 1 TO 3

 MOVEABS x y

 DRAWTEXT "+"

 x = x + .5

 NEXT

 y = y - 1.5

NEXT

PRIVATE f AS INTEGER

PRIVATE s AS STRING

FILEOPEN(f, "myfile.txt", "w")

FILEWRITELN("LaserStar", f)

FILEWRITELN("Technologies", f)

FILEWRITELN("Corporation", f)

FILECLOSE(f)

PRIVATE var(100) AS STRING

FILEOPEN(f, "myfile.txt", "r")

WHILE (FILEREADln(var(i),f))

 HMIMESSAGE var(i)

 i = i + 1

ENDWHILE


```

;Example 4
UNITSINCH
TEXTHEIGHT .1

DEFPROG 1, "Classes.fmp"

alpha.set("Test")
if (alpha.value.left(4) = "TEST")
    alpha.value = "LaserStar"
    HMIMESSAGE alpha.value
else
    HMIMESSAGE ("Try Left(4) = " + "\"" + "Test" + "\")
endif

DEFPROG 2, "MoreClasses.fmp"

DRAWTEXT "\"" + "HelloWorld" + "\""

delta.set("More")
HMIMESSAGE delta.value

theta.set("Test")
HMIMESSAGE theta.value

EXIT

```

```

; "Classes.fmp"

alpha.value = "A"

beta.value = "B"

gamma.value = "C"

HMIMESSAGE alpha.value
HMIMESSAGE beta.value
HMIMESSAGE gamma.value

private s AS STRING
s = alpha.get
HMIMESSAGE s
s = beta.get
HMIMESSAGE s
s = gamma.get
HMIMESSAGE s
alpha.set("X")

beta.set("Y")

gamma.set("Z")

CLASS alpha
    PRIVATE value AS STRING
    FUNCTION set(BYVAL newvalue AS STRING)
        value = newvalue
    ENDFUNCTION
    FUNCTION get() AS STRING
        RETURN value
    ENDFUNCTION
ENDCLASS
CLASS beta
    PRIVATE value AS STRING
    FUNCTION set(BYVAL newvalue AS STRING)
        value = newvalue
    ENDFUNCTION
    FUNCTION get() AS STRING
        RETURN value
    ENDFUNCTION
ENDCLASS
CLASS gamma
    PRIVATE value AS STRING
    FUNCTION set(BYVAL newvalue AS STRING)
        value = newvalue
    ENDFUNCTION
    FUNCTION get() AS STRING
        RETURN value
    ENDFUNCTION
ENDCLASS

```



```
;”moreclasses.fmp”

CLASS delta
  PRIVATE value AS STRING
  FUNCTION set(BYVAL newvalue AS STRING)
    value = newvalue
  ENDFUNCTION
  FUNCTION get() AS STRING
    RETURN value
  ENDFUNCTION
ENDCLASS

CLASS theta
  PRIVATE value AS STRING
  FUNCTION set(BYVAL newvalue AS STRING)
    value = newvalue
  ENDFUNCTION
  FUNCTION get() AS STRING
    RETURN value
  ENDFUNCTION
ENDCLASS
```